

CareerLens AI — System Documentation

Comprehensive Technical Reference, Architecture Pipeline & Testing Manual

1. Executive Summary & Objective

CareerLens AI is an intelligent, multi-modal career readiness and technical interview evaluation platform. Modern job seekers frequently struggle with generic interview preparation tools that fail to interrogate the candidate's actual projects, while recruiters and interviewers struggle to verify whether claimed skills and resumes represent genuine hands-on competence.

The primary objective of CareerLens AI is to provide an authentic, adaptive, and automated interview simulation that:

- **Audits Resumes with Transparent NLP:** Parses candidate resumes to evaluate ATS (Applicant Tracking System) compatibility, detect 7 skill clusters, and identify missing sections without black-box opacity.
- **Validates Project Authenticity:** Automatically extracts claimed project names from the candidate's resume and conducts targeted architecture and roadblock probing in every interview.
- **Generates Resume-Aware Interviews:** Replaces one-size-fits-all static question sets with personalized questions derived specifically from candidate-claimed languages, tools, and experiences.
- **Engages in Intelligent Conversational Follow-Ups:** Analyzes the candidate's spoken responses in real-time, extracts mentioned technical concepts, and asks natural, name-addressed follow-up questions connecting to advanced technical domains.
- **Drills CS Fundamentals:** Offers dedicated interactive drills with instant evaluation and model solutions across Operating Systems, DBMS, Networks, OOP, and DSA.
- **Enforces Academic Integrity:** Incorporates a client-side proctoring engine with full-screen enforcement, key restriction, audio warning synthesizer, and automated cancellation upon repeated violations.

2. System Architecture & Tech Stack

CareerLens AI is built with a modular, lightweight Python architecture designed to run seamlessly in both local environments and resource-constrained cloud containers (e.g. Streamlit Community Cloud):

Tier	Technology	Role & Architecture Details
Presentation & UI	Streamlit 1.37+	Reactive single-page web framework; manages session states, audio capture widgets, progress meters, and dynamic tabs.
Styling System	Vanilla CSS & Glassmorphism	Custom dark-themed visual design with CSS grid cards, SVG circular score indicators, and smooth micro-animations.
Speech Recognition	faster-whisper (tiny/int8)	Quantized CTranslate2 implementation of OpenAI Whisper running locally on CPU (~150MB footprint) for voice transcription.
Speech Synthesis	HTML5 Web Speech API	Zero-latency, client-side browser voice synthesizer supporting American, British, and Indian English accents.
NLP & Parsing	spaCy, NLTK, PyMuPDF	Local tokenization, lemmatization, named-entity recognition, PDF binary stream extraction, and OCR image reading.

Generative AI / LLM	Google Gemini 2.5 Flash	Hybrid LLM integration (google-genai / REST API) for dynamic follow-up generation, project authenticity probing, and answer critique.
Proctoring Sandbox	Vanilla JS + Web Audio	Browser full-screen event listener, shortcut key interception, synthesizer beeps, and security query parameter state machine.

3. Detailed Library Breakdown & Purpose

Each Python library in CareerLens AI was chosen specifically to balance cutting-edge generative AI capabilities with zero-cost offline resiliency:

Library	Specific Feature / Purpose	Implementation Location
google-genai	Google Gemini 2.5 Flash SDK for contextual follow-up question generation, project authenticity probing, and coach feedback.	utils/llm_engine.py
streamlit	Core application server, state management (session_state), user interface rendering, and live audio inputs.	app.py
nltk	Sentence tokenization (sent_tokenize), word segmentation (word_tokenize), stopword filtering, and lemmatization (WordNetLemmatizer).	utils/nlp_engine.py
spacy	Named Entity Recognition (extracting organizations, institutions, names, dates), and linguistic noun-chunk extraction for adaptive follow-ups.	utils/nlp_engine.py utils/nlp_features.py
scikit-learn	TfidfVectorizer and cosine_similarity for mathematical resume-to-job-description textual alignment scoring.	utils/nlp_features.py
PyMuPDF (fitz)	Direct in-memory PDF text extraction without saving temporary files to disk, supporting multi-page candidate resumes.	utils/resume_parser.py
Pillow & pytesseract	Optical Character Recognition (OCR) fallback for candidates uploading scanned document images (JPG/PNG).	utils/resume_parser.py
faster-whisper	High-performance speech-to-text inference on CPU with automatic Voice Activity Detection (VAD) to filter background noise.	utils/speech.py
av (PyAV)	Direct audio demuxing and decoding from web browser WebM audio containers to PCM arrays for Whisper.	utils/speech.py

4. Core Methodologies & Algorithms

4.1 Project Authenticity Probing Algorithm

To verify whether a candidate genuinely authored the projects listed on their resume, the platform employs a two-tier extraction and probing pipeline:

1. **Section & Heading Demarcation:** Scans for PROJECTS, ACADEMIC PROJECTS, or TECHNICAL PROJECTS section headers, isolating content lines preceding the next standard section.
2. **Title Disambiguation:** Filters out operational bullet points (lines beginning with action verbs like *Developed*, *Implemented*, *Built*) and identifies clean project names (e.g. *'Smart Attendance System'*) while extracting associated tech stack tokens.
3. **Realistic Skill-Based Fallback:** If no explicit project heading is present, an intelligent inference engine constructs a realistic project title based on the candidate's detected skills (e.g., Python + ML yields *'AI-Powered Predictive Analytics System'*).
4. **Dynamic Question Formulation:** Generates a mandatory Q1 interview question explicitly citing the project name: *'So [Name], on your resume you highlighted your project [Project Name]... Could you walk me through the end-to-end architecture, your personal contributions, and the single hardest bug you encountered?'*

4.2 Intelligent Concept-Aware Follow-Up Engine

Instead of asking rigid or generic follow-ups, CareerLens AI inspects the candidate's transcribed spoken answer in real time:

- **Linguistic Concept Identification:** Matches spoken words against a technical ontology (e.g., *'Artificial Intelligence'*, *'Deep Learning'*, *'React'*, *'SQL'*, *'Docker'*).
- **Dual Formulation Modes:** Alternates between: (a) *Related Concept Pivot* (e.g., *'So Adarsh, as you mentioned Artificial Intelligence, do you know about deep learning and neural networks?'*) and (b) *Under-the-Hood Probe* (e.g., *'As you mentioned Artificial Intelligence, can you explain what it is and how algorithms learn patterns from data?'*).
- **Dynamic spaCy Noun-Chunk Fallback:** If the concept is outside the static knowledge base, spaCy extracts the root noun phrase from the candidate's response to construct an organic follow-up.

4.3 Hybrid Generative AI & Resilient Local Fallback Engine

CareerLens AI integrates a dual-tier question generation and answer evaluation engine:

- **Primary Tier (Google Gemini 2.5 Flash):** When a Gemini API key is configured (via Streamlit Secrets or sidebar), the platform prompts the LLM with strict system instructions to generate single-sentence, highly contextual follow-ups, project authenticity probes, and 2-sentence coach critiques.
- **Resilient Secondary Fallback:** If no API key is supplied, network connectivity drops, or quota is reached, the system automatically falls back to its internal technical ontology and spaCy NLP pipeline with zero runtime errors or interruption.

4.4 Mathematical Resume ↔ Job Description Matching

The system calculates text similarity using TF-IDF (Term Frequency-Inverse Document Frequency) and cosine distance:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times \ln[(1 + |D|) / (1 + DF(t, D))] + 1$$

$$\text{Cosine Similarity} = (V_{\text{resume}} \cdot V_{\text{jd}}) / (||V_{\text{resume}}|| \times ||V_{\text{jd}}||)$$

The computed angle between vectors yields an exact match percentage (0–100%), while set difference operations between the JD vocabulary and resume tokens highlight missing ATS keywords.

4.5 Full-Screen Anti-Cheating & Keyboard Sandboxing

A client-side security harness enforces strict proctoring during the interview:

- **Fullscreen Lock:** Hooks into `document.documentElement.requestFullscreen()`. Exiting fullscreen immediately increments the warning counter.
- **Shortcut Interception:** Intercepts keydown events, suppressing default browser actions for Alt+Tab, Ctrl+Tab, Ctrl+C, Ctrl+V, Ctrl+F, F12, PrintScreen, Win, Esc, Alt+F4, Ctrl+W, and Ctrl+L.
- **Synthetic Audio Beeps:** Uses the Web Audio API (`AudioContext`) to play a dual-tone alert tone (880Hz / 440Hz) when a violation occurs.
- **Four-Strike Termination:** Upon the 4th infraction, the interview is cancelled, the session is cleared, and an urgent cancellation banner is displayed.

5. Complete Step-by-Step Application Pipeline

Step 1: Ingestion & Upload: The user uploads a PDF or image resume (or pastes raw text) and enters their name on the Home dashboard.

Step 2: NLP Analysis & Profiling: PyMuPDF/pytesseract extracts the text. The NLP pipeline performs sentence tokenization, stopword cleaning, lemmatization, and regex matching across 7 technical categories. Entities (persons, organizations, universities) are extracted via spaCy NER.

Step 3: ATS Score Calculation: A deterministic ATS scoring engine evaluates the profile against 10 critical criteria (contact channels, section completeness, skill density, formatting), outputting an ATS Score (0–100%) and an actionable checklist.

Step 4: Job Description Matching (Optional): The candidate pastes a target Job Description. The TF-IDF cosine similarity engine produces a match percentage and lists specific missing technical keywords.

Step 5: Interview Configuration: Candidate selects their target company (Microsoft, Google, Amazon, Deloitte, Cognizant, Wipro, TCS, JP Morgan, Cisco, or Custom JD), interview mode (Practice or 60s Timed), and voice accent.

Step 6: Interview Orchestration: The sequence starts with Q0 (Introduction), followed by Q1 (Project Authenticity Probe citing candidate's project name), Q2+ (Resume-Aware Questions + Company Questions).

Step 7: Real-Time Audio & Evaluation: Candidate records answers via the browser audio input. faster-whisper transcribes speech; NLP evaluation checks expected concepts, filler words, fluency, and generates automatic follow-up questions.

Step 8: Final Diagnostic Report: Upon completion, the application aggregates performance metrics: overall score, concept coverage, vocabulary diversity, filler word analysis, and specific improvement recommendations.

6. Complete Step-by-Step Testing Manual

This section provides quality assurance procedures to test every module of CareerLens AI thoroughly:

TC-01: Resume Upload & Text Extraction (PDF / OCR)

Testing Steps:	1. Navigate to '[Home]'. 2. Upload sample PDF (e.g. sample_resume/software.pdf) or paste raw text. 3. Enter candidate name (e.g. 'Adarsh Nayak'). 4. Click 'Analyze Resume'.
Expected Result:	Extracted text populates the preview box; ATS Score card displays with breakdown; detected skills and contact details appear with green checkmarks.

TC-02: Resume <-> Job Description TF-IDF Matcher

Testing Steps:	1. In the JD section, paste a job posting requiring Python, Docker, and AWS. 2. Click 'Calculate Match Score'.
Expected Result:	Match percentage is computed; matched keywords are shown in green; missing keywords appear with recommendations.

TC-03: Project Authenticity Probing Verification

Testing Steps:	1. Ensure resume contains a project title (e.g. 'Smart Attendance System'). 2. Select any company and start the interview. 3. Complete Q0 (Introduction). 4. Observe Q1 prompt.
Expected Result:	Q1 explicitly cites the project: 'So Adarsh, on your resume you highlighted your project 'Smart Attendance System'...' with cyan '[PROJECT PROBE]' badge.

TC-04: Intelligent Concept Follow-Up Verification

Testing Steps:	1. In Q1, record or type an answer mentioning 'Artificial Intelligence'. 2. Click 'STOP / SUBMIT'. 3. Observe next question.
Expected Result:	System automatically generates Q1.1 addressing candidate: 'So Adarsh, as you mentioned Artificial Intelligence, do you know about deep learning...' with yellow '[FOLLOW-UP]' badge.

TC-05: Resume-Aware Question Verification

Testing Steps:	1. Upload a resume listing React, Docker, and Machine Learning. 2. Start an interview with 5 questions. 3. Inspect questions Q2 and Q3.
Expected Result:	Questions directly reference candidate's detected skills: 'So Adarsh, your resume mentions React...' with purple '[RESUME-AWARE]' badge.

TC-06: CS Core Subject Practice & Drill Verification

Testing Steps:	1. Navigate to '[CS Core]' tab. 2. Filter by subject (e.g. 'Operating Systems') and difficulty. 3. Click '[Random Question]'. 4. Type answer and click 'Submit Answer'. 5. Click 'Reveal Model Answer'.
Expected Result:	System evaluates concept coverage, relevance, and shows complete expert model solution.

TC-07: Fullscreen Proctoring & Key Restriction Testing

Testing Steps:	1. Start an interview. 2. Click '[Enter Fullscreen Mode]'. 3. Press forbidden keys: Alt+Tab, Ctrl+C, F12, or Esc.
Expected Result:	Default browser actions are blocked; red warning modal appears; warning count increments from 1/4 to 2/4; synthetic warning beep sounds.

TC-08: Proctoring Cancellation Enforcement

Testing Steps:	1. Trigger 4 proctoring infractions during an active interview session.
Expected Result:	On the 4th infraction, interview terminates immediately; full-screen exits; '[URGENT WARNING: Your interview has been CANCELLED]' banner is displayed.

TC-09: Voice Transcription & Interview Report

Testing Steps:	1. Record answer via browser microphone. 2. Verify transcription matches spoken words. 3. Complete all questions in the interview.
Expected Result:	System generates comprehensive Diagnostic Report with overall score, vocabulary richness, filler word analysis, and strengths/weaknesses.

TC-10: Google Gemini LLM Follow-Up & Local Fallback Testing

Testing Steps:	1. In Sidebar, enter a Gemini API Key (or leave blank for offline test). 2. Answer a question mentioning a technical tool or framework. 3. Observe follow-up question format and UI badge.
Expected Result:	If API key is valid, badge displays pink '[AI FOLLOW-UP]' with highly contextual probing. If key is missing or offline, badge displays yellow '[ADAPTIVE FOLLOW-UP]' via local NLP fallback with zero crashes.

7. Cloud Deployment & Troubleshooting Guide

CareerLens AI is packaged for continuous deployment to **Streamlit Community Cloud** (share.streamlit.io). The root repository contains three automated deployment descriptors:

- **requirements.txt:** Specifies exact pre-built Python wheels. Standardizes on en-core-web-sm @ URL syntax to ensure installation during the container build step without runtime permission errors.
- **nltk.txt:** Automates NLTK corpora provisioning (punkt, stopwords, wordnet) during image build.
- **packages.txt:** Instructs the Debian package manager to install tesseract-ocr and ffmpeg for OCR and audio processing.